

AI Recruiter Ranker

A CPU-only, explainable system for ranking Senior AI Engineer candidates

- Ranks 100,000 profiles into a trusted top-100 shortlist.
- Optimized for the Redrob founding Senior AI Engineer JD.
- No hosted APIs, no GPU, no hidden manual edits during ranking.

What the JD really asks for

The model ranks for evidence, not keyword density

- Strongest signal: shipped search, retrieval, recommendation, ranking, NLP or ML systems.
- Must be a hands-on product engineer, not a pure researcher or demo-only framework user.
- 5-9 years is ideal, with Pune/Noida or nearby Indian hubs preferred.
- Availability matters: recent activity, recruiter response, short notice and open-to-work improve trust.

Ranking architecture

Streaming feature extraction plus deterministic hybrid scoring

- 1. Stream candidates.jsonl so memory stays small.
- 2. Build structured features from profile, career history, skills and Redrob behavioral signals.
- 3. Score five components: career fit, retrieval/eval, production/product context, logistics, engagement.
- 4. Apply penalties for nontechnical profiles, service-only histories, inactive candidates and honeypot-like inconsistencies.
- 5. Emit ranked CSV/XLSX with specific reasoning for every shortlisted candidate.

Why this works against traps

Career-history evidence beats skill stuffing

- Skills are only supporting evidence; role titles and descriptions carry more weight.
- HR, marketing, content and design profiles are penalized unless career text proves real ML/search work.
- Expert skills with near-zero duration and keyword-heavy profiles with weak career evidence are down-ranked.
- Service-only histories are discounted unless the candidate has strong retrieval/ranking or product evidence.

Score design

Weighted to mimic how a strong recruiter would shortlist

- 45% career evidence and role-title fit.
- 20% retrieval, search, recommendation and evaluation evidence.
- 15% production and product-company context.
- 10% experience, location, relocation and notice-period logistics.
- 10% engagement and availability signals, minus explicit risk penalties.

Reproducibility

Simple command, fast offline runtime

- Command: `python scripts/rank_candidates.py --candidates candidates.jsonl --out deliverables/submission.csv --xlsx deliverables/submission.xlsx`
- Implementation uses the Python standard library for ranking and XLSX export.
- The same code creates the submitted top-100; no candidate data is sent to any LLM/API.
- Local validation checks row count, rank order, candidate existence, monotonic scores and reasoning.

Deliverables

What is included

- GitHub repo: clean source, README, build plan, validation script and metadata template filled.
- Ranked output: CSV for the official spec and XLSX for the portal upload field.
- Deck PDF: this approach summary.
- Demo path: Streamlit-ready app skeleton can rank uploaded small samples using the same scorer.